Recursion : Function calling itself.

$$sum(N) = \underline{1 + 2 + 3 + \cdots N-1} + N$$

$$sum(N) = sum(N-1) + N. \rightarrow \text{Main logic.}$$

* In recursion, the bigger problem is broken down into subproblems and the result of the subproblems is used to compute the result of the current problem.

How to write recursive code?

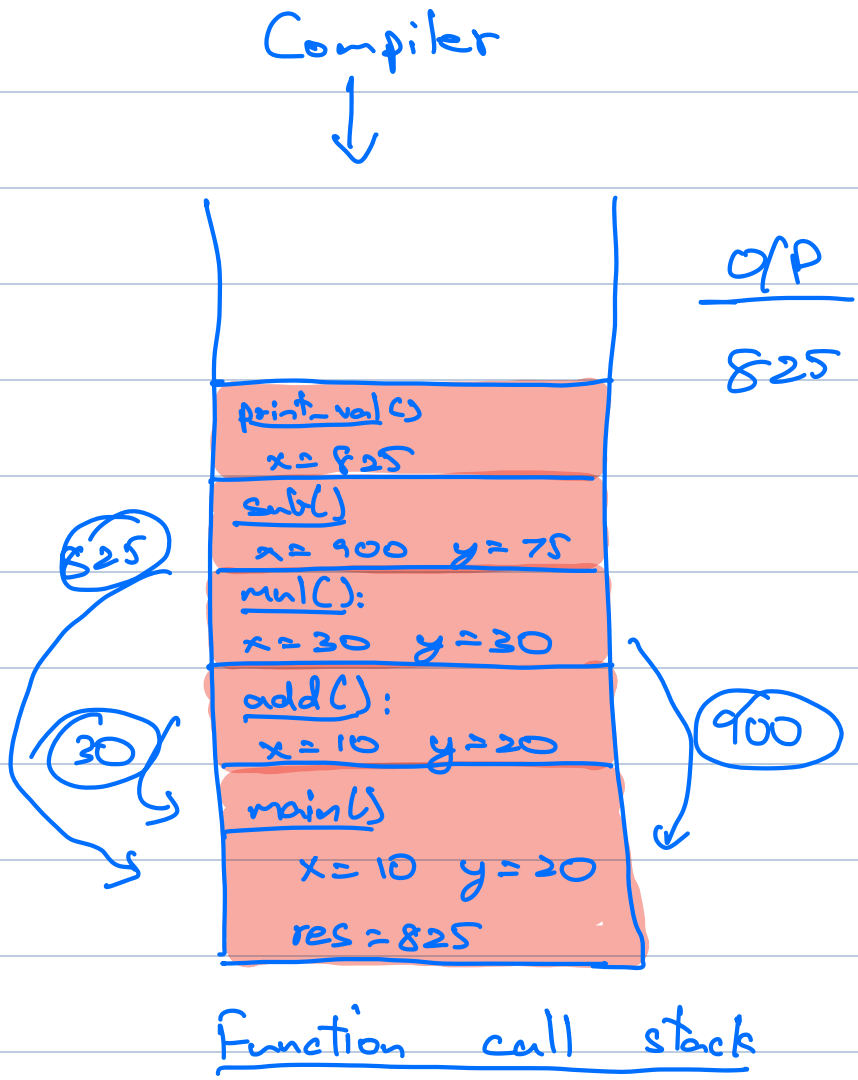
1) Assumption: Assume that the subproblem is solved.
2) Main logic: Use the subproblem's result to compute the current problem's result.
3) Base case(s): Stop case where recursive calls end.

$$sum(3) = \underline{sum(2)} + 3$$
$$\downarrow$$
$$\underline{sum(1)} + 2$$
$$\downarrow$$
$$\underline{sum(0)} + 1$$
$$\downarrow$$
$$\underline{sum(-1)} + 0$$
$$\downarrow$$
$$sum(-2) + (-1)$$
$$\downarrow$$
$$\cdots$$

```
def sumN(N):
    # Base Case(s)
    if(N==0) return 0.
    # Main logic
    return sumN(N-1)+N
```

Function Call Tracing

```
def add(x,y):
    return x+y
def mul(x,y):
    return x*y
def sub(x,y):
    return x-y
def print_value(x):
    print(x)
def main():
    x,y = 10, 20
    res = sub(mul(add(x,y), 30), 75)
    print_value(res)
```



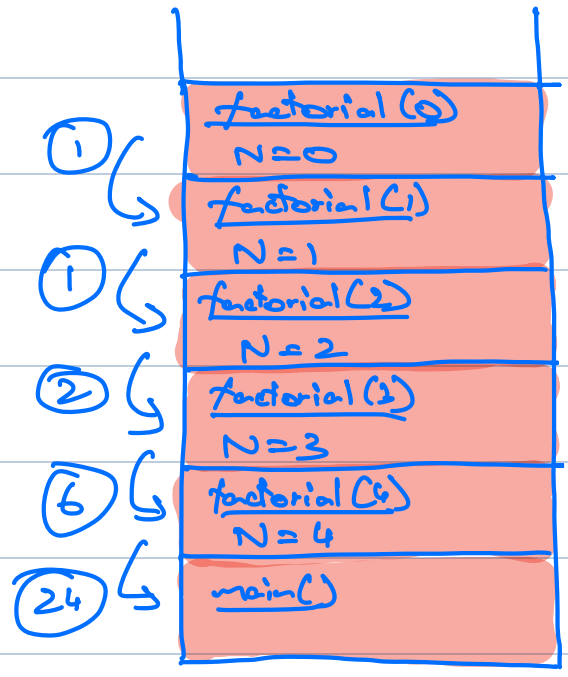
Question: Given a positive number N, find the factorial of N.

$$N! = \underline{1 * 2 * 3 * \ldots (N-1)} * N.$$

factorial(N) = factorial(N-1) * N

ex: $5! = 1 * 2 * 3 * 4 * 5 = 120.$

```
def factorial(N):
    # Base Case(s)
    if (N==0) : return 1
    # Main Logic.
    return factorial(N-1)*N
```



factorial(4) # 5 calls # 5 depth. Call stack

TC = O(# of func^n calls * Time per func^n call)

$= O((N+1) * 1) = O(N)$

SC = O(Max Depth of stack + Space per func^n call).

$= O((N+1) + 1) = O(N)$

Question: Given positive N, print all numbers from 1 to N in increasing order.

ex: N = 5 → 1 2 3 4 5

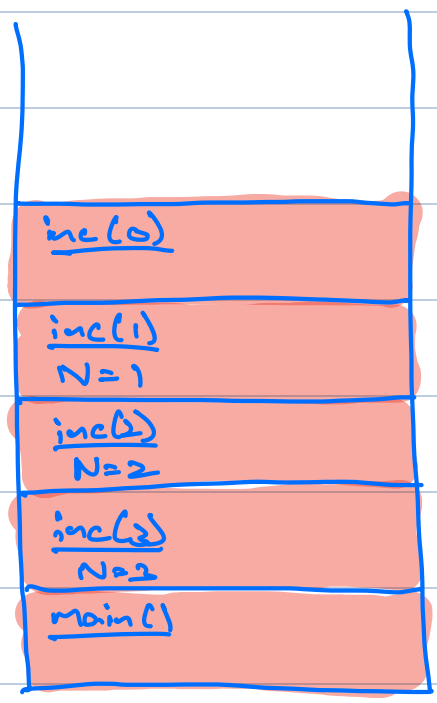
inc(N) ⇒ 1 2 3 . . . N-1 N

inc(N) ⇒ inc(N-1) + print(N)

↳ Main Logic

Base Case: N==0 ⇒ return None.

```
def inc(N):
    # Base Case(s)
    if N==0 : return
    # Main logic
    inc(N-1)
    print(N)
inc(3)
```



O/P

1

2

3

TC = O(# of func^n calls * Time per func call)

= O((N+1) * 1) = O(N)

SC = O(Max depth + Space per func^n call).

= O((N+1) + 1) = O(N).

Question: Given positive N, print all numbers from N to 1. in decreasing order.

dec (N) → N $\underbrace{N-1 \quad N-2 \quad \cdots \quad 1}$

dec (N) → print(N) + dec(N-1)

↳ Main logic

Base Case: N==0 → return.

```
def dec(N):
    # Base Case(s)
    if (N==0) : return
    # Main logic
    print(N)
    dec(N-1)
```

O/P

3

2

1

dec(3)

TC = O(# of func^n calls * Time per func call)

= O((N+1) * 1) = O(N)

SC = O(Max depth * Space per func^n call).

= O((N+1) * 1) = O(N)

Time Complexity and Space Complexity

TC = O(# of func^n calls * Time per func^n call)

SC = O(Max depth of stack + Space per func^n call)

Question: Given a positive integer N, compute the Nth number in the Fibonacci Series.

N:	0	1	2	3	4	5	6	7	8
f(N):	0	1	1	2	3	5	8	13	21

ex: N = 6

O/P = 8

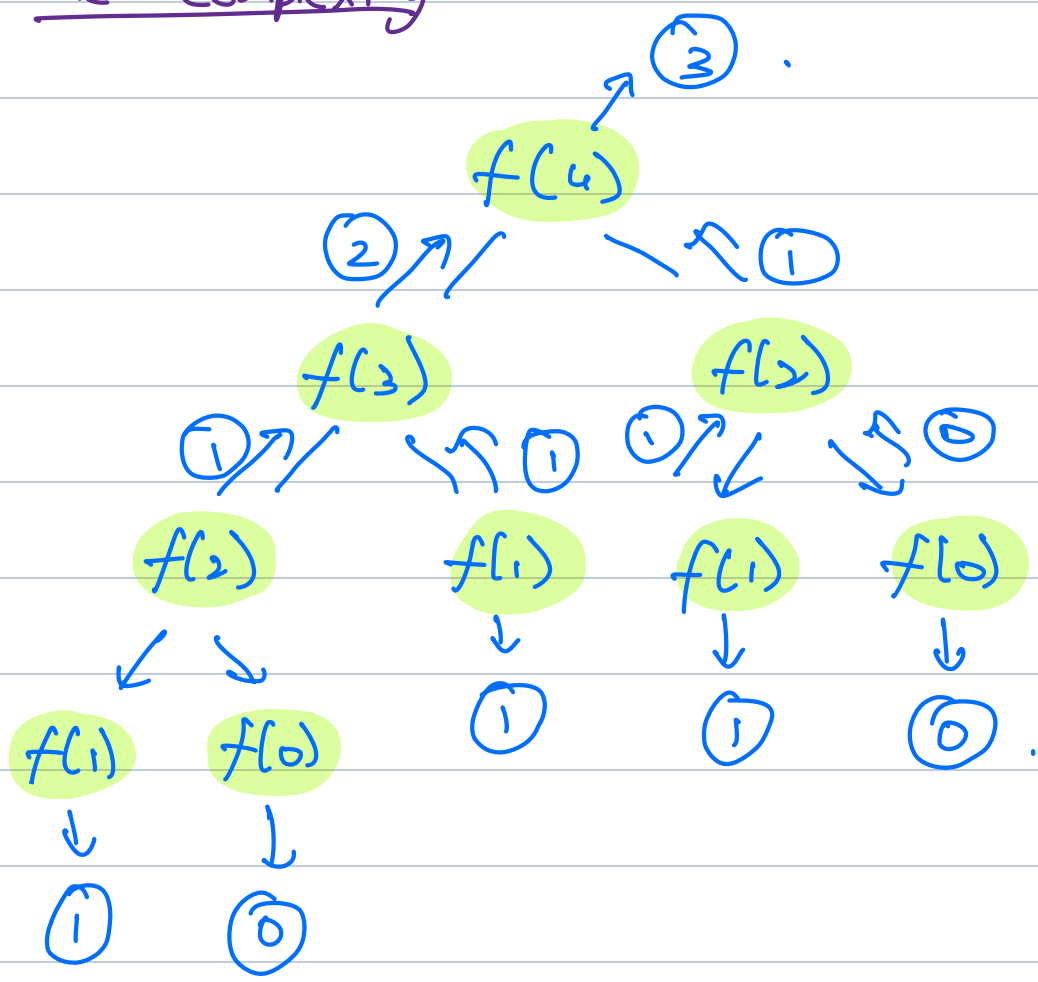
fibonacci(N) = fibonacci(N-1) + fibonacci(N-2)

↳ Main logic

Base Case(s): if(N <= 1): return N.

```
def fibonacci(N):
    # Base Case(s)
    if (N <= 1): return N.
    # Main logic
    return fibonacci(N-1) + fibonacci(N-2).
```

Time Complexity



Func^n calls

$\rightarrow$ 1

+

$\rightarrow$ 2

+

$\rightarrow$ 4

+

$\rightarrow$ 8

⋮

Levels

$$\underbrace{N \rightarrow N-1 \rightarrow N-2 \rightarrow \cdots \cdot 1}_{N \text{ levels}}$$

Func^n calls

$$\underbrace{1 + 2 + 4 + 8 + \cdots}_{N \text{ levels/terms.}} \quad \text{GP.}$$

$$\text{Sum} = \frac{a * (r^n - 1)}{r - 1} = \frac{1 * (2^N - 1)}{2 - 1} = (2^N - 1)$$

$TC = O(\#\text{ of func}^n \text{ calls} * \text{Time per func}^n \text{ call})$

$= O((2^N - 1) * 1)$

$\boxed{TC = O(2^N)}$

$SC = O(\text{Max Depth} + \text{Space per func}^n \text{ call})$.

$= O(N + 1)$

$\boxed{SC = O(N)}$